GUÍA OPERATIVA · SANDBOX

Despliegue automatizado y solución de problemas en el Sandbox

DocuProcess Cloud · Guía operativa para AWS Academy Learner Lab

PROYECTO	DocuProcess Cloud
CURSO	Infraestructura III · Universidad ICESI
AUTORES	Daniela Castaño Moreno y Alejandro Pérez García
VERSIÓN	1.0 · 13 de noviembre de 2026
REPOSITORIO	github.com/Alejandroperezgarc1a/docuprocess-cloud
REGIÓN	us-east-1

NAVEGACIÓN

Contenido

Resumen: qué se despliega y cuánto tarda 3

1. Requisitos y restricciones del Sandbox 4

2. Credenciales y preflight 6

3. Configuración del proyecto 9

4. Despliegue automatizado 11

5. Verificación de extremo a extremo 14

6. Operación, actualización y destrucción 19

7. Solución de problemas 21

8. Contraste con el enunciado y la rúbrica 26

9. Anexos 29

Los números de página corresponden al PDF. La guía rápida *despliegue.md* sigue disponible como referencia de bolsillo.

> Esta guía es el procedimiento operativo para desplegar DocuProcess Cloud desde cero en el Sandbox de AWS Academy, verificar el flujo completo y resolver los fallos que aparecen con más frecuencia. La guía rápida de referencia sigue disponible en `despliegue.md`; este documento la amplía con preflight, importación de recursos huérfanos, verificación por capas y contraste explícito con el enunciado.

Resumen: qué se despliega y cuánto tarda

DocuProcess Cloud es un SaaS de procesamiento asíncrono de documentos. El usuario entra a un portal estático servido por **CloudFront + S3**, se autentica contra una **API REST** que corre en **EC2** detrás de un **ALB**, sube archivos directamente a **S3** con URL prefirmada y un **worker** en la misma instancia procesa el documento, guarda metadatos e historial en **DynamoDB** y notifica por **SNS**. Las cuentas y sesiones viven en **RDS MySQL**. La infraestructura se crea con **Terraform modular** y el laboratorio se opera sin SSH, usando **SSM Session Manager**.

Resultado esperado	Cómo se comprueba
Infraestructura creada desde una plantilla limpia	`terraform apply` termina sin errores y el plan ronda los **73 recursos** con `monitoring`
Frontend y API publicados en un solo dominio HTTPS	`app_url` responde `/api/health` y `/api/ready` con HTTP 200
Flujo S3 → worker → DynamoDB → SNS	Un PDF real termina en `DONE`; un archivo corrupto termina en `FAILED`; llega el correo
Capacidad elástica	La prueba de carga lleva el ASG de 2 a 4 instancias y luego vuelve a 2
Monitoreo y auditoría	4 alarmas en `OK`, acciones hacia SNS y eventos visibles en CloudTrail

Duración realista: 30–45 minutos para el primer despliegue completo, más el tiempo de confirmación del correo SNS. La creación de RDS, CloudFront y el reemplazo inicial del ASG es la parte más lenta; no es un proceso de 15 minutos. Si el laboratorio ya trae recursos huérfanos, agrega 5–15 minutos de diagnóstico e importación (§2.4 y §7).

Ruta rápida

#	Paso	Comando principal	Tiempo aproximado
1	Iniciar Lab y cargar credenciales	`export AWS_PROFILE=academy`	2 min
2	Verificar cuenta, región y rol SSM	`aws sts get-caller-identity`	1 min
3	Revisar estado de Terraform	`terraform -chdir=infra state list`	1–2 min
4	Configurar variables y secretos	`cp infra/terraform.tfvars.example infra/terraform.tfvars`	3 min
5	Validar y empaquetar	`terraform -chdir=infra validate` y `./scripts/package-app.sh`	3–5 min
6	Aplicar infraestructura	`terraform -chdir=infra apply`	25–40 min
7	Publicar frontend	`./scripts/deploy-frontend.sh`	3–5 min
8	Confirmar SNS y verificar E2E	correo + `curl` + navegador	5 min

1. Requisitos y restricciones del Sandbox

1.1 Herramientas locales

Herramienta	Versión mínima	Verificación	Para qué se usa
AWS CLI v2	2.x	`aws --version`	Credenciales, consultas, SSM, SNS, CloudWatch
Terraform	1.9	`terraform version`	Crear y destruir la infraestructura
Node.js	22 (24 recomendado)	`node -v`	Compilar API, worker y frontend
npm	10	`npm -v`	Dependencias y build
Git	2.x	`git --version`	Clonar el repositorio
jq	1.6	`jq --version`	Procesar respuestas JSON
curl	7.x	`curl --version`	Verificar API y subir archivos
zip / unzip	3.x	`zip -v`	Empaquetar el artefacto de las instancias
OpenSSL	1.1	`openssl version`	Generar secretos
ApacheBench (`ab`)	2.3	`ab -V`	Prueba de carga del ASG (opcional)
Python 3	3.10	`python3 --version`	Utilidades locales y verificación de documentos

`ab` solo es necesario para la prueba de carga. En Debian/Ubuntu se instala con `sudo apt-get install -y apache2-utils` ; en Fedora/RHEL, con `sudo dnf install -y httpd-tools` .

1.2 Restricciones del Sandbox y cómo las cumple el proyecto

Restricción del enunciado	Decisión en DocuProcess Cloud	Dónde se evidencia
IAM de solo lectura; no `CreateRole` ni `GetRole`	No se crean roles. Se descubre el instance profile preexistente con `data.aws_iam_instance_profiles` y se usan políticas basadas en recursos	`infra/main.tf` , `docs/trazabilidad.md` fila 12
Lambda no disponible	El worker corre como servicio systemd en las mismas instancias EC2 que la API	`scripts/user-data.sh.tftpl` , `docs/architecture.md`
EC2 `t2/t3` nano–medium; máximo 9 concurrentes; EBS `gp2` ≤ 35 GB	`t3.micro` , ASG 2–4, volumen raíz `gp2` de 20 GB y validaciones en variables	`infra/variables.tf` , `infra/modules/compute/`
RDS `db.t2/t3` micro–medium, Single-AZ, `gp2` ≤ 100 GB	MySQL 8.0 `db.t3.micro` , Single-AZ, 20 GB `gp2` , sin réplica	`infra/modules/rds/`
Sin dominios personalizados en Route 53	Se usan los DNS por defecto del ALB y de CloudFront	`infra/outputs.tf` (`alb_dns_name` , `cloudfront_domain_name`)
IaC con Terraform o CloudFormation	Terraform modular con 8 módulos	`infra/main.tf`
Sin SSH ni bastión	SSM Session Manager con el rol preexistente `AmazonSSMRoleForInstancesQuickSetup`	`infra/modules/compute/` , §5.6
Credenciales temporales del Learner Lab	El despliegue se ejecuta desde la estación con un perfil de AWS CLI; no hay credenciales en el repositorio	§2

Restricción del enunciado	Decisión en DocuProcess Cloud	Dónde se evidencia
Algunas acciones están denegadas por la SCP	Se documentan fallbacks: `event-history`, sin `instance_refresh`, sin Object Lock, transacciones desactivables	§3.4 y §7

> **Importante:** el Sandbox no es un entorno de producción. No se debe prometer alta disponibilidad real: RDS es Single-AZ por restricción, CloudFront usa su certificado por defecto y el estado de Terraform es local. La guía explica estas salvedades en §8.4.

1.3 Qué significa "despliegue automatizado" en este proyecto

El proyecto automatiza la **construcción y la aplicación** de la infraestructura, no la ejecución remota de un pipeline de CD:

- **IaC:** un solo `terraform apply` crea red, cómputo, datos, CDN, monitoreo y parámetros.
- **Scripts idempotentes:** `package-app.sh` genera el artefacto versionado por hash; `deploy-frontend.sh` valida cuenta y outputs antes de escribir; `empty-bucket.sh` limpia versiones y multipart uploads.
- **CI de validación:** GitHub Actions verifica formato, `terraform validate`, typecheck, build, tests, `shellcheck`, gitleaks y el empaquetado determinista.
- **Sin CD a AWS:** el Sandbox entrega credenciales temporales y no permite crear el rol OIDC que un runner necesitaría. Por eso el `apply` y la demo se ejecutan manualmente con las credenciales del laboratorio. Esto es una decisión consciente, no un olvido.

Job de CI	Qué valida
`detect`	Cambios en infraestructura, backend, frontend o scripts
`secrets`	Gitleaks sobre el repositorio
`terraform`	`fmt -check -recursive`, `init -backend=false`, `validate`
`node`	API y worker en `amazonlinux:2023` con Node 24: instalación, typecheck, build y tests
`frontend`	Build de Vite con Node 24
`shellcheck`	Scripts y userData renderizado en sus dos variantes
`package`	Zip determinista y verificación de contenido

1.4 Checklist antes de tocar AWS

- ☐ Tengo una sesión de Learner Lab iniciada y las credenciales vigentes.
- ☐ `aws sts get-caller-identity` devuelve la cuenta correcta.
- ☐ La región configurada es `us-east-1`.
- ☐ Existe el instance profile `AmazonSSMRoleForInstancesQuickSetup`.
- ☐ Tengo un correo real para `alert_email` y puedo confirmar el mensaje de SNS.
- ☐ Node.js es 22 o superior; idealmente 24.
- ☐ No hay un `terraform.tfstate` de otra cuenta en `infra/`.
- ☐ Entiendo que el primer `apply` puede tardar 30–45 minutos.

2. Credenciales y preflight

2.1 Iniciar el laboratorio y configurar el perfil

1. En AWS Academy, entra al curso y pulsa **Start Lab**. Espera a que el indicador esté en verde.
2. Abre **AWS Details** → **AWS CLI** → **Show** y copia `aws_access_key_id`, `aws_secret_access_key` y `aws_session_token`.
3. Configura un perfil local. No escribas las credenciales en el repositorio:

```
aws configure --profile academy
# AWS Access Key ID:     pegar la del laboratorio
# AWS Secret Access Key: pegar la del laboratorio
# Default region name:   us-east-1
# Default output format: json

export AWS_PROFILE=academy
export AWS_REGION=us-east-1
```

También puedes editar `~/.aws/credentials` y `~/.aws/config` a mano:

```
# ~/.aws/credentials
[academy]
aws_access_key_id = ...
aws_secret_access_key = ...
aws_session_token = ...

# ~/.aws/config
[profile academy]
region = us-east-1
output = json
```

> El `aws_session_token` es obligatorio en Learner Lab. Si lo omites, verás errores de autenticación aunque las claves parezcan correctas.

2.2 Verificar identidad, región y rol SSM

```
aws sts get-caller-identity --output table
aws configure get region
aws iam list-instance-profiles \
  --query "InstanceProfiles[?
InstanceProfileName=='AmazonSSMRoleForInstancesQuickSetup'].InstanceProfileName" \
  --output text
```

Resultado esperado:

- La cuenta es un número de 12 dígitos y el ARN corresponde a un rol asumido del laboratorio.
- La región es `us-east-1`.
- El comando de instance profiles imprime `AmazonSSMRoleForInstancesQuickSetup`.

Si el último comando no imprime nada, lista todos los perfiles y ajusta `instance_role_name` en `terraform.tfvars`:

```
aws iam list-instance-profiles \
  --query 'InstanceProfiles[].InstanceProfileName' \
  --output table
```

> En algunas sesiones las APIs de lectura (`Describe*` , `GetMetricStatistics` , `LookupEvents`) están denegadas por la SCP. Si un comando de verificación falla con `AccessDenied` , usa la consola de AWS o continúa con los comandos que sí estén permitidos; no interpretes ese fallo como un error del despliegue.

2.3 Verificar el estado de Terraform

Entra al repositorio y revisa si hay estado local:

```
git clone git@github.com:Alejandroperezgarc1a/docuprocess-cloud.git
cd docuprocess-cloud

terraform -chdir=infra init -input=false
terraform -chdir=infra state list
terraform -chdir=infra plan -refresh=true
```

Interpretación:

Resultado	Qué significa	Acción
`No state file was found` y plan con ~73 recursos	Laboratorio sin estado de Terraform	Continuar con §3 y §4
Plan con `No changes`	La infraestructura coincide con el estado	Ir a verificación (§5)
Plan con cambios inesperados	El laboratorio cambió o el estado es de otra sesión	Revisar §2.4 antes de aplicar
Error de credenciales	La sesión expiró o el perfil es incorrecto	Renovar credenciales (§2.5)

> **Nunca** ejecutes `terraform apply` con un plan que destruye recursos que no reconoces. En un Sandbox compartido, el estado local puede apuntar a una sesión anterior.

2.4 El laboratorio no siempre está limpio: recursos huérfanos

La evidencia de C4 (13-nov-2026) demuestra que un laboratorio “nuevo” puede conservar recursos: RDS `docuprocess-dev-mysql` , los parámetros SSM `/docuprocess-dev/db_password` y `/docuprocess-dev/jwt_secret` , y el OAC de CloudFront. El primer `apply` falló con `ParameterAlreadyExists` y `OriginAccessControlAlreadyExists` ; se resolvió importando los tres recursos y volviendo a aplicar.

Detecta huérfanos antes de aplicar:

```
# RDS
aws rds describe-db-instances \
  --query "DBInstances[?DBInstanceIdentifier=='docuprocess-dev-mysql'].
[DBInstanceIdentifier,DBInstanceStatus,DBInstanceClass]" \
  --output table

# Parámetros SSM
aws ssm get-parameters \
  --names /docuprocess-dev/db_password /docuprocess-dev/jwt_secret \
```

```
  --query 'Parameters[].{Name:Name,Version:Version,Type:Type}' \
  --output table

# OAC de CloudFront
aws cloudfront list-origin-access-controls \
  --query "OriginAccessControlList.Items[?contains(Name,'docuprocess')].[Id,Name]" \
  --output table

# Buckets con nombre del proyecto
aws s3api list-buckets \
  --query "Buckets[?contains(Name,'docuprocess')].Name" \
  --output table
```

Si `terraform state list` está vacío pero los recursos existen, importa solo los que el plan intentaría crear. Los tres casos verificados en C4 son:

```
terraform -chdir=infra import aws_ssm_parameter.db_password /docuprocess-dev/db_password
terraform -chdir=infra import aws_ssm_parameter.jwt_secret /docuprocess-dev/jwt_secret
terraform -chdir=infra import 'module.cdn.aws_cloudfront_origin_access_control.frontend' <OAC_ID>
```

Sustituye `<OAC_ID>` por el valor de la primera columna del comando de OAC. Si el estado ya contiene el recurso, Terraform responderá que ya está administrado: no lo importes dos veces. Después del import:

```
terraform -chdir=infra plan -refresh=true
```

El plan debe bajar a los recursos que realmente faltan. En C4 pasó de un `apply` fallido a **7 to add, 0 to change, 0 to destroy** y luego a un `apply` exitoso.

> Si aparece un RDS huérfano, no lo borres a ciegas: puede contener las cuentas de una demo anterior. Impórtalo con `terraform -chdir=infra import module.rds.aws_db_instance.this docuprocess-dev-mysql` o elimínalo de forma consciente. La guía prioriza reutilizarlo.

2.5 Si la sesión expira

Síntomas típicos: `InvalidClientTokenId`, `ExpiredToken`, `The security token included in the request is expired` o un `403` repentino en comandos que antes funcionaban.

```
# 1. Volver a AWS Academy y copiar credenciales nuevas.
# 2. Actualizar el perfil.
aws configure --profile academy

# 3. Verificar y refrescar el estado.
aws sts get-caller-identity
terraform -chdir=infra plan -refresh=true
```

No hace falta destruir nada. Terraform refresca el estado con las credenciales nuevas. Si el laboratorio cambió de cuenta, el estado anterior ya no sirve: sigue §6.6.

3. Configuración del proyecto

3.1 Crear el archivo de variables

```
cp infra/terraform.tfvars.example infra/terraform.tfvars
```

Edita `infra/terraform.tfvars` con tus datos. El archivo está cubierto por `*.tfvars` en `.gitignore`; no lo fuerces con `git add -f`.

Ejemplo mínimo:

```
aws_region   = "us-east-1"
project_name = "docuprocess"
environment  = "dev"

alert_email = "tu-correo@example.com"

# Genera estos valores con openssl; no reutilices los de ejemplo.
db_password = "REEMPLAZAR_CON_OPENSSL_RAND_HEX_16"
jwt_secret  = "REEMPLAZAR_CON_OPENSSL_RAND_HEX_32"
```

3.2 Generar secretos válidos

```
openssl rand -hex 16   # db_password: 32 caracteres ASCII, sin / @ "
openssl rand -hex 32   # jwt_secret:  64 caracteres
```

Pega cada salida en `terraform.tfvars`. La validación del proyecto exige que `db_password` tenga al menos 12 caracteres ASCII y que no contenga `/`, `@` ni `"`; `jwt_secret` debe tener al menos 32 caracteres.

> Los secretos quedan en el `terraform.tfstate` local y en los parámetros SSM. No compartas el estado, no lo subas al repositorio y bórralo después de destruir el laboratorio (§6.5).

3.3 Variables que más importan

Variable	Valor por defecto	Cuándo cambiarla
`alert_email`	— (obligatoria)	Siempre; debe ser un correo que puedas confirmar
`db_password`	— (obligatoria)	Siempre; usar `openssl rand -hex 16`
`jwt_secret`	— (obligatoria)	Siempre; usar `openssl rand -hex 32`
`aws_region`	`us-east-1`	No cambiar en el Sandbox
`instance_type`	`t3.micro`	Solo si el laboratorio lo permite y respeta el límite
`asg_min_size` / `asg_max_size`	`2` / `4`	Mantener 2–4 para la demo y el límite de 9 instancias
`asg_desired_capacity`	`2`	Mantener 2 antes de la prueba de carga
`root_volume_size`	`20` GB	Nunca por encima de 35 GB (`gp2`)
`db_name` / `db_username`	`docuprocess` / `docuadmin`	Solo si hay una razón; no afecta la demo

Variable	Valor por defecto	Cuándo cambiarla
`enable_dynamodb_transactions`	`true`	Poner `false` si la SCP deniega escrituras transaccionales
`enable_point_in_time_recovery`	`true`	Poner `false` si el Sandbox deniega PITR
`cloudtrail_mode`	`event-history`	Dejar así; `trail` requiere permisos que el Sandbox deniega
`force_destroy_buckets`	`true`	Dejar así para poder destruir el laboratorio
`docs_lifecycle_expiration_days`	`0`	`0` desactiva la expiración; útil en demo

3.4 Flags de contingencia

El Sandbox puede denegar acciones que sí están disponibles en una cuenta normal. El proyecto ya trae los fallbacks:

Si falla...	Cambia a...	Efecto
`dynamodb:TransactWriteItems` o escrituras transaccionales	`enable_dynamodb_transactions = false`	La API deja de usar transacciones y hace las escrituras por separado
`dynamodb:UpdateContinuousBackups` / PITR	`enable_point_in_time_recovery = false`	Se desactiva PITR; las tablas siguen funcionando
`cloudtrail:CreateTrail`	`cloudtrail_mode = "event-history"`	Se usa el historial de eventos de 90 días, sin trail persistente
`autoscaling:StartInstanceRefresh`	No hay variable: el módulo no usa `instance_refresh`	El reemplazo se hace creando un ASG nuevo o terminando instancias de una en una
`s3:GetBucketObjectLockConfiguration`	El módulo ya usa `awscc_s3_bucket`	Evita la llamada denegada al crear buckets

Después de cambiar un flag:

```
terraform -chdir=infra plan
terraform -chdir=infra apply
```

4. Despliegue automatizado

4.1 Validar la infraestructura antes de crear nada

```
terraform -chdir=infra fmt -check -recursive
terraform -chdir=infra init -input=false
terraform -chdir=infra validate
```

Si `fmt -check` falla, formatea y vuelve a validar:

```
terraform -chdir=infra fmt -recursive
terraform -chdir=infra validate
```

4.2 Empaquetar el backend

El artefacto debe existir **antes** del `apply`. Terraform lo sube a S3 con un nombre que incluye su hash; si el hash cambia, el launch template cambia y el ASG reemplaza instancias.

```
./scripts/package-app.sh
ls -lh dist/docuprocess-app.zip
```

El script exige Node.js 22 o superior, compila API y worker, instala dependencias de producción y genera un zip determinista. En las instancias se usa Node.js 24 instalado por `dnf`.

> Si cambias código de `app/api` o `app/worker`, vuelve a ejecutar `package-app.sh` antes de cada `apply`. Si no lo haces, Terraform puede reutilizar el artefacto anterior.

4.3 Plan y apply

```
terraform -chdir=infra plan -out=tfplan
terraform -chdir=infra apply tfplan
rm -f infra/tfplan
```

El plan muestra el resumen. Con el estado vacío y el módulo de monitoreo, el orden de magnitud es **~73 recursos**; el número exacto depende de los recursos huérfanos que se importen y de las opciones activas. No compares el resultado con el "68" histórico de la primera integración: ese número no incluía todas las alarmas actuales.

Durante el `apply` verás, entre otros:

1. VPC, subredes, route tables, gateway endpoints e interface endpoints.
2. Buckets S3, tablas DynamoDB, topic SNS y parámetros SSM.
3. RDS MySQL Single-AZ y su subnet group.
4. ALB, target group, launch template y ASG.
5. Distribución CloudFront, OAC y bucket policy del frontend.
6. Alarmas CloudWatch y, si aplica, CloudTrail.

4.4 Tiempos esperados

Etapa	Tiempo típico	Nota
`terraform init`	1–2 min	Descarga providers `aws`, `awscc` y `random`
`package-app.sh`	2–5 min	Depende de npm y de la caché local
`terraform plan`	1–3 min	Más lento si hay que refrescar RDS/CloudFront
`terraform apply`	25–40 min	RDS, CloudFront y ASG dominan el tiempo
`deploy-frontend.sh`	3–5 min	Build de Vite, sync e invalidación
Confirmación SNS	manual	Depende de que llegue el correo

4.5 Revisar outputs

```
terraform -chdir=infra output
terraform -chdir=infra output -raw app_url
terraform -chdir=infra output -raw alb_dns_name
terraform -chdir=infra output -raw asg_name
terraform -chdir=infra output -raw sns_topic_arn
```

Output	Uso
`app_url`	URL HTTPS de CloudFront para la demo
`alb_dns_name`	URL directa del ALB para diagnóstico
`asg_name`	Consultas de capacidad y reemplazo de instancias
`sns_topic_arn`	Verificar suscripción y alarmas
`rds_endpoint`	Comprobar RDS (no expone la contraseña)
`cloudfront_distribution_id`	Invalidaciones manuales
`account_id`	Confirmar que el estado pertenece a la cuenta actual

4.6 Publicar el frontend

```
./scripts/deploy-frontend.sh
```

El script:

1. Verifica AWS CLI, Terraform y Node.js ≥ 22.12.
2. Compara la cuenta de AWS con el output `account_id` del estado; falla antes de escribir si no coinciden.
3. Lee `frontend_bucket_name`, `cloudfront_distribution_id` y `cloudfront_domain_name`.
4. Genera `frontend/public/config.js` con `apiBaseUrl: ""` (mismo dominio, sin CORS).
5. Ejecuta `npm ci` y `npm run build`.
6. Sincroniza assets con caché inmutable e `index.html` / `config.js` sin caché.
7. Crea una invalidación `/*` y espera a que termine.
8. Imprime la URL final.

Abre `app_url` en el navegador. Si ves una pantalla en blanco, revisa §7.

4.7 Confirmar la suscripción de SNS

Terraform crea la suscripción, pero **no puede confirmarla**: AWS envía un correo al `alert_email` y la suscripción queda en `PendingConfirmation` hasta que alguien hace clic.

```
TOPIC_ARN="$(terraform -chdir=infra output -raw sns_topic_arn)"
aws sns list-subscriptions-by-topic \
  --topic-arn "$TOPIC_ARN" \
  --query 'Subscriptions[].{Protocol:Protocol,Endpoint:Endpoint,Arn:SubscriptionArn}' \
  --output table
```

Si `SubscriptionArn` dice `PendingConfirmation`, busca el correo "AWS Notification - Subscription Confirmation" y confirma. Vuelve a ejecutar el comando: debe aparecer un ARN completo.

> Sin esta confirmación no llegan ni las alertas de documentos ni las de CloudWatch. Es el olvido más común en la demo.

4.8 Re-desplegar después de un cambio

Backend o worker:

```
./scripts/package-app.sh
terraform -chdir=infra plan -out=tfplan
terraform -chdir=infra apply tfplan
rm -f infra/tfplan
```

El nuevo hash del artefacto cambia el launch template y el ASG se reemplaza. El módulo usa `create_before_destroy` y no usa `instance_refresh` porque la SCP deniega `autoscaling:StartInstanceRefresh`. Durante el reemplazo puede haber instancias temporales adicionales; con 2–4 deseadas y un máximo de 9 concurrentes hay margen.

Solo frontend:

```
./scripts/deploy-frontend.sh
```

Cambio de variables sin cambio de código:

```
terraform -chdir=infra plan
terraform -chdir=infra apply
```

5. Verificación de extremo a extremo

5.1 Checklist rápido

- ☐ `aws sts get-caller-identity` sigue funcionando.
- ☐ `terraform -chdir=infra plan` no muestra cambios inesperados.
- ☐ `app_url/api/health` responde 200.
- ☐ `app_url/api/ready` responde 200 con MySQL `ok`.
- ☐ El target group tiene 2/2 targets `healthy`.
- ☐ El ASG tiene 2/2 instancias `InService`.
- ☐ La suscripción SNS está confirmada (ARN completo).
- ☐ Registro, login y creación de documento funcionan.
- ☐ Un PDF termina en `DONE` y un corrupto en `FAILED`.
- ☐ Llega el correo de SNS del documento.
- ☐ Las 4 alarmas están en `OK` (o `INSUFFICIENT_DATA` recién creadas).
- ☐ CloudTrail muestra eventos recientes.
- ☐ La prueba de carga lleva el ASG a 4 y luego baja a 2.

5.2 Red, ALB y ASG

```
ALB_DNS="$(terraform -chdir=infra output -raw alb_dns_name)"
curl -sS -o /dev/null -w "ALB health=%{http_code}\n" "http://$ALB_DNS/api/health"

TG_ARN="$(aws elbv2 describe-target-groups \
  --names docuprocess-dev-app \
  --query 'TargetGroups[0].TargetGroupArn' \
  --output text)"
aws elbv2 describe-target-health \
  --target-group-arn "$TG_ARN" \
  --query 'TargetHealthDescriptions[].
{Target:Target.Id,State:TargetHealth.State,Reason:TargetHealth.Reason}' \
  --output table

ASG="$(terraform -chdir=infra output -raw asg_name)"
aws autoscaling describe-auto-scaling-groups \
  --auto-scaling-group-names "$ASG" \
  --query 'AutoScalingGroups[0].
{Min:MinSize,Max:MaxSize,Desired:DesiredCapacity,Instances:Instances[].
{Id:InstanceId,Lifecycle:LifecycleState,Health:HealthStatus}}' \
  --output table
```

Esperado: ALB 200, dos targets `healthy`, ASG `Min=2`, `Max=4`, `Desired=2`, dos instancias `InService` / `Healthy`. Si el target group no se llama `docuprocess-dev-app`, usa `aws elbv2 describe-target-groups --query 'TargetGroups[].TargetGroupName'`.

5.3 API y CloudFront

```
APP_URL="$(terraform -chdir=infra output -raw app_url)"
curl -sS -o /dev/null -w "CloudFront health=%{http_code}\n" "$APP_URL/api/health"
curl -sS "$APP_URL/api/ready" | jq .
```

Esperado:

```
{"status":"ready","checks":{"mysql":"ok"}}
```

`/api/health` no toca la base: sirve para saber si la API está viva. `/api/ready` comprueba MySQL: si devuelve 503, el problema está en RDS, en el security group o en las credenciales.

5.4 Datos: RDS, DynamoDB y S3

```
aws rds describe-db-instances \
  --db-instance-identifier docuprocess-dev-mysql \
  --query 'DBInstances[0].
{Status:DBInstanceStatus,Class:DBInstanceClass,MultiAZ:MultiAZ,Storage:AllocatedStorage,Engine:Engin
e}' \
  --output table

aws dynamodb describe-table \
  --table-name "$(terraform -chdir=infra output -raw dynamodb_table_name)" \
  --query 'Table.{Status:TableStatus,Items:ItemCount,Billing:BillingModeSummary.BillingMode}' \
  --output table

aws s3 ls "s3://$(terraform -chdir=infra output -raw docs_bucket_name)/uploads/" | head
```

Esperado: RDS `available`, `db.t3.micro`, `MultiAZ=False`, 20 GB; DynamoDB `ACTIVE`; el prefijo `uploads/` existe cuando ya se subió al menos un documento.

5.5 Flujo E2E con `curl`

El siguiente bloque registra un usuario, obtiene un JWT, crea un documento, sube un PDF con la URL prefirmada y consulta el estado hasta `DONE`.

```
APP_URL="$(terraform -chdir=infra output -raw app_url)"
EMAIL="demo-$(date +%s)@example.com"
PASSWORD='DemoPass123!'
FILE="/ruta/a/tu/documento.pdf"          # cambia esta ruta
SIZE="$(stat -c%s "$FILE")"               # macOS: stat -f%z "$FILE"

# 1. Registro
curl -sS -X POST "$APP_URL/api/auth/register" \
  -H 'Content-Type: application/json' \
  -d "{\"email\":\"$EMAIL\",\"password\":\"$PASSWORD\"}" | jq .

# 2. Login y token
TOKEN="$(curl -sS -X POST "$APP_URL/api/auth/login" \
  -H 'Content-Type: application/json' \
  -d "{\"email\":\"$EMAIL\",\"password\":\"$PASSWORD\"}" | jq -r '.token')"
echo "token_len=${#TOKEN}"

# 3. Crear documento y obtener URL prefirmada
CREATE="$(curl -sS -X POST "$APP_URL/api/documents" \
  -H "Authorization: Bearer $TOKEN" \
```

```
  -H 'Content-Type: application/json' \
  -d "{\"filename\":\"documento.pdf\",\"contentType\":\"application/pdf\",\"size\":$SIZE}")"
echo "$CREATE" | jq .
DOC_ID="$(echo "$CREATE" | jq -r '.documentId')"
UPLOAD_URL="$(echo "$CREATE" | jq -r '.uploadUrl')"

# 4. Subir el archivo directamente a S3
curl -sS -X PUT "$UPLOAD_URL" \
  -H 'Content-Type: application/pdf' \
  --data-binary "@$FILE" \
  -o /dev/null -w "upload=%{http_code}\n"

# 5. Consultar el estado hasta DONE o FAILED
for i in $(seq 1 30); do
  STATUS="$(curl -sS "$APP_URL/api/documents/$DOC_ID" \
    -H "Authorization: Bearer $TOKEN" | jq -r '.status')"
  echo "status=$STATUS"
  case "$STATUS" in
    DONE|FAILED) break ;;
  esac
  sleep 5
done

# 6. Historial de eventos
curl -sS "$APP_URL/api/documents/$DOC_ID/events" \
  -H "Authorization: Bearer $TOKEN" | jq .
```

Resultados esperados:

- Registro: `201` con `id` y `email`.
- Login: `200` con `token` y `expiresIn: 3600`.
- Creación: `201` con `documentId`, `uploadUrl`, `method: "PUT"`, `headers` y `expiresIn: 900`.
- Upload: `200`.
- Estado: `PENDING → PROCESSING → DONE`.
- Eventos: al menos `PENDING`, `PROCESSING` y `DONE` con fechas.

Prueba negativa (archivo corrupto):

```
printf 'esto no es un pdf\n' > /tmp/corrupt.pdf
# Repite los pasos 3–5 con filename="corrupto.pdf",
# contentType="application/pdf" y size del archivo corrupto.
# Resultado esperado: FAILED.
```

Prueba de aislamiento: crea un segundo usuario y trata de leer el documento del primero con `GET /api/documents/$DOC_ID`. Debe devolver `404` o `401`, nunca el contenido ajeno.

5.6 Worker y logs por SSM

```
ASG="$(terraform -chdir=infra output -raw asg_name)"
INSTANCE_ID="$(aws autoscaling describe-auto-scaling-groups \
  --auto-scaling-group-names "$ASG" \
  --query 'AutoScalingGroups[0].Instances[0].InstanceId' \
  --output text)"

CMD_ID="$(aws ssm send-command \
  --instance-ids "$INSTANCE_ID" \
  --document-name AWS-RunShellScript \
```

```
  --parameters 'commands=["systemctl is-active docuprocess-api docuprocess-worker","systemctl status
docuprocess-api --no-pager -n 20","sudo tail -n 40 /var/log/docuprocess-userdata.log"]' \
  --query 'Command.CommandId' --output text)"

sleep 5
aws ssm get-command-invocation \
  --command-id "$CMD_ID" --instance-id "$INSTANCE_ID" \
  --query '{Status:Status,Output:StandardOutputContent,Error:StandardErrorContent}' \
  --output json
```

Esperado: `active` para `docuprocess-api` y `docuprocess-worker`; el log de userData sin errores de descarga del artefacto ni de instalación de Node.

Sesión interactiva:

```
aws ssm start-session --target "$INSTANCE_ID"
# Dentro de la instancia:
sudo journalctl -u docuprocess-api -u docuprocess-worker -n 100 --no-pager
sudo tail -n 100 /var/log/docuprocess-userdata.log
```

No hay puerto 22 abierto ni clave SSH: todo el acceso es por SSM.

5.7 SNS, alarmas y CloudTrail

```
TOPIC_ARN="$(terraform -chdir=infra output -raw sns_topic_arn)"
aws sns list-subscriptions-by-topic \
  --topic-arn "$TOPIC_ARN" \
  --query 'Subscriptions[].{Protocol:Protocol,Endpoint:Endpoint,Arn:SubscriptionArn}' \
  --output table

aws cloudwatch describe-alarms \
  --alarm-name-prefix docuprocess-dev- \
  --query 'MetricAlarms[].{Name:AlarmName,State:StateValue,Metric:MetricName,Threshold:Threshold}' \
  --output table

./scripts/verify-cloudtrail.sh 10
```

Esperado:

- Suscripción con ARN completo (no `PendingConfirmation`).
- Cuatro alarmas: `docuprocess-dev-ec2-cpu-high`, `docuprocess-dev-rds-cpu-high`, `docuprocess-dev-rds-connections-high`, `docuprocess-dev-rds-free-storage-low`.
- Estados `OK`; `INSUFFICIENT_DATA` es normal justo después de crearlas.
- CloudTrail devuelve eventos recientes en `us-east-1` (modo `event-history`, sin trail).

Para demostrar una alarma sin esperar una carga real, baja temporalmente el umbral de `docuprocess-dev-ec2-cpu-high` a 1 %, espera dos periodos de 5 minutos, recibe el correo y restaura el valor a 80 %. No dejes ese cambio en la entrega.

5.8 Prueba de carga del ASG

```
command -v ab >/dev/null || sudo apt-get install -y apache2-utils
mkdir -p logs
LOG_DIR=./logs ./scripts/load-test.sh
```

El script registra un usuario único, valida el login, ejecuta `ab` durante 600 s con 50 concurrentes contra `/api/auth/login` del ALB y muestra cada 30 s la capacidad deseada, las instancias `InService` y la CPU del ASG. Guarda los logs en `LOG_DIR`.

Mientras corre, puedes observar el ASG en otra terminal:

```
ASG="$(terraform -chdir=infra output -raw asg_name)"
watch -n 30 "aws autoscaling describe-auto-scaling-groups \
  --auto-scaling-group-names '$ASG' \
  --query 'AutoScalingGroups[0].{Desired:DesiredCapacity,InService:length(Instances)}' \
  --output table"
```

Evidencia esperada (A8, 13-nov-2026): CPU 98–99 % desde +162 s, ASG de 2 a 4 a los +333 s, 600 s, 50 concurrentes, 5.848 requests, 32 fallidos y 4 respuestas no-2xx transitorias. El scale-in a 2 tarda unos 15 minutos porque la alarma de baja necesita varios periodos con CPU baja. No interpretes esos 15 minutos como un fallo.

5.9 Lectura de resultados

Señal	Significado	Acción
`/api/health` 200 y `/api/ready` 503	API viva, MySQL no disponible	Revisar RDS, security group y credenciales
Target group con targets `unhealthy`	La API no responde en el puerto 8000	SSM + `journalctl`; revisar userData y artefacto
Documento `PROCESSING` por más de 5 min	Worker detenido o error de procesamiento	§7.6
Documento `FAILED` con PDF válido	Tipo/contenido rechazado o error del parser	Revisar eventos y logs del worker
`PendingConfirmation` en SNS	Nadie confirmó el correo	Confirmar y repetir
Alarma en `INSUFFICIENT_DATA`	Aún no hay métricas suficientes	Esperar 2 periodos; es normal al inicio
ASG no sube con carga	La prueba no llega al ALB o la política no tiene métricas	Revisar `load-test.sh`, target group y CPU

6. Operación, actualización y destrucción

6.1 Re-desplegar backend o worker

```
./scripts/package-app.sh
terraform -chdir=infra plan -out=tfplan
terraform -chdir=infra apply tfplan
rm -f infra/tfplan
```

Espera a que el ASG reemplace las instancias y el target group vuelva a 2/2 `healthy`. No ejecutes la prueba de carga durante el reemplazo: mezclarías CPU de arranque con la métrica de escalado.

6.2 Re-desplegar solo el frontend

```
./scripts/deploy-frontend.sh
```

Si cambiaste `config.js` o `index.html`, la invalidación `/*` ya se encarga de que CloudFront no sirva la versión anterior. Los assets con hash se pueden cachear de forma inmutable.

6.3 Rotar secretos

1. Cambia `db_password` o `jwt_secret` en `terraform.tfvars`.
2. Aplica:

```
terraform -chdir=infra plan
terraform -chdir=infra apply
```

1. Terraform crea una nueva versión de los parámetros SSM. Las instancias **no** leen el secreto en caliente: lo hacen en el userData al arrancar. Para que lo tomen, reemplaza las instancias de una en una:

```
ASG="$(terraform -chdir=infra output -raw asg_name)"
INSTANCE_ID="$(aws autoscaling describe-auto-scaling-groups \
  --auto-scaling-group-names "$ASG" \
  --query 'AutoScalingGroups[0].Instances[0].InstanceId' --output text)"
aws autoscaling terminate-instance-in-service \
  --instance-id "$INSTANCE_ID" \
  --no-should-decrement-desired-capacity
```

Espera a que el ASG reponga la instancia y quede `Healthy` antes de repetir con la otra. No cambies `db_password` sin actualizar también la contraseña real de RDS: el proyecto usa la misma variable para crear RDS y para el parámetro SSM, pero un cambio manual en RDS no se sincroniza solo.

6.4 Reemplazar instancias sin `instance_refresh`

El módulo no usa `instance_refresh` porque la SCP del Sandbox deniega `autoscaling:StartInstanceRefresh`. Las opciones seguras son:

- **Cambio de launch template:** `package-app.sh` cambia el hash del artefacto; Terraform crea un ASG nuevo con nombre `...-app-v<versión>` y elimina el anterior con `create_before_destroy`.

- **Terminación controlada:** termina una instancia `InService` y espera el reemplazo antes de tocar la siguiente.
- **Destruir y recrear:** solo para una demo desde cero; no es un despliegue sin caída.

6.5 Destruir y limpiar

```
terraform -chdir=infra plan -destroy
terraform -chdir=infra destroy
```

El módulo de storage usa `terraform_data.empty_buckets` y `scripts/empty-bucket.sh` para vaciar objetos, versiones, delete markers y multipart uploads antes de borrar los buckets. Si el `destroy` falla porque un bucket no está vacío:

```
DOCS_BUCKET="$(terraform -chdir=infra output -raw docs_bucket_name)"
FRONTEND_BUCKET="$(terraform -chdir=infra output -raw frontend_bucket_name)"
ARTIFACTS_BUCKET="$(terraform -chdir=infra output -raw artifacts_bucket_name)"

./scripts/empty-bucket.sh "$DOCS_BUCKET"
./scripts/empty-bucket.sh "$FRONTEND_BUCKET"
./scripts/empty-bucket.sh "$ARTIFACTS_BUCKET"
terraform -chdir=infra destroy
```

Después de confirmar que no quedan recursos:

```
terraform -chdir=infra state list
rm -f infra/terraform.tfstate infra/terraform.tfstate.backup infra/tfplan
```

> No borres el estado antes de que el `destroy` termine: Terraform lo necesita para saber qué recursos eliminar. Si el laboratorio se apaga solo, el estado puede quedar apuntando a recursos que ya no existen; en ese caso usa `terraform state rm` con cuidado o empieza con un estado nuevo.

6.6 Cambiar de sesión o de cuenta

Si `terraform output -raw account_id` no coincide con `aws sts get-caller-identity`, **no apliques**. Mueve el estado a un lado y empieza de cero:

```
mv infra/terraform.tfstate infra/terraform.tfstate.old-account
terraform -chdir=infra init -input=false
terraform -chdir=infra plan
```

El estado viejo no se puede usar para destruir recursos de otra cuenta. Si necesitas limpiar la cuenta anterior, hazlo con credenciales de esa cuenta.

7. Solución de problemas

7.1 Árbol de decisión

1. **¿Falla `terraform init`?** Credenciales, red o providers. Revisa §2.5 y `terraform version`.
2. **¿Falla `plan`?** Variables inválidas, estado de otra cuenta o recursos huérfanos. Revisa §2.4 y §3.
3. **¿Falla `apply`?** Lo más común es un recurso preexistente (`ParameterAlreadyExists`, `OriginAccessControlAlreadyExists`) o una acción denegada por la SCP. Revisa §7.3 y §7.4.
4. **¿El `apply` termina pero la app no responde?** ALB/target group, userData, artefacto o SSM. Revisa §7.5.
5. **¿La app responde pero el upload falla?** CORS, URL prefirmada expirada o `Content-Type`. Revisa §7.7.
6. **¿El upload funciona pero el documento no termina?** Worker, logs, DynamoDB o S3. Revisa §7.6.
7. **¿Todo funciona pero no llegan correos?** SNS sigue en `PendingConfirmation` o la política del topic no permite la acción. Revisa §7.8.

7.2 Tabla de casos frecuentes

Síntoma	Causa probable	Qué hacer	Prevención
`InvalidClientTokenId` / `ExpiredToken`	Credenciales del Lab expiradas	Volver a copiar credenciales y `terraform plan -refresh=true`	Renovar antes de cada demo
`AccessDenied` en casi todo	Perfil equivocado o cuenta incorrecta	`aws sts get-caller-identity`; comparar con `terraform output account_id`	Exportar `AWS_PROFILE` en cada terminal
`ParameterAlreadyExists`	Parámetros SSM huérfanos	Importar (§2.4) o borrar los parámetros	Preflight de huérfanos
`OriginAccessControlAlreadyExists`	OAC de una sesión anterior	Importar `module.cdn...frontend` con su ID	Preflight de OAC
`DBInstanceAlreadyExists`	RDS huérfano	Importar `module.rds.aws_db_instance.this` o eliminar el RDS	Preflight de RDS
`dist/docuprocess-app.zip` no existe	No se ejecutó el empaquetado	`./scripts/package-app.sh`	Empaquetar antes de cada apply
Instancias `unhealthy` o userData falla	Artefacto, Node, SSM o permisos	SSM + `/var/log/docuprocess-userdata.log`; §7.5	Verificar 2/2 healthy antes de la demo
CloudFront 504 intermitente	ALB en draining durante reemplazo	Esperar 1–2 min y reintentar; revisar target group	No probar durante un reemplazo
SNS en `PendingConfirmation`	Correo no confirmado	Confirmar el correo; revisar spam	Confirmar antes de la demo
Documento atascado en `PROCESSING`	Worker caído o excepción	SSM + `journalctl`; reiniciar el worker; §7.6	Verificar worker activo

Síntoma	Causa probable	Qué hacer	Prevención
`503` al crear documento	Transacciones DynamoDB denegadas	`enable_dynamodb_transactions = false` y aplicar	Tener el flag listo
PUT prefirmado `403`	URL expirada, header o CORS	Regenerar URL, usar `Content-Type` exacto, revisar CORS	Subir en menos de 15 min
Pantalla en blanco	Frontend sin build o `config.js` viejo	`./scripts/deploy-frontend.sh`; revisar consola	Publicar frontend después del apply
`npm ERR! engine`	Node < 22	`node -v`; usar Node 24	Fijar Node 24
`ab: command not found`	Falta ApacheBench	Instalar `apache2-utils` o `httpd-tools`	Instalarlo antes de A8
`destroy` falla por bucket no vacío	Versiones o multipart uploads	`./scripts/empty-bucket.sh <bucket>` y repetir	`force_destroy_buckets=true`
`cloudtrail:CreateTrail` denegado	SCP del Sandbox	Usar `cloudtrail_mode="event-history"`	No prometer trail persistente
`StartInstanceRefresh` denegado	SCP del Sandbox	No usar `instance_refresh`; reemplazar por ASG/terminación	Diseño ya previsto

7.3 Recursos huérfanos e importación

El error típico es:

```
Error: creating SSM Parameter (...): ParameterAlreadyExists: The parameter already exists.
```

No borres el parámetro sin mirar: puede ser el que usan instancias vivas. Si el estado está vacío y el recurso pertenece al proyecto, impórtalo:

```
terraform -chdir=infra import aws_ssm_parameter.db_password /docuprocess-dev/db_password
terraform -chdir=infra import aws_ssm_parameter.jwt_secret /docuprocess-dev/jwt_secret
```

Para el OAC:

```
aws cloudfront list-origin-access-controls \
  --query "OriginAccessControlList.Items[?Name=='docuprocess-dev-frontend-oac'].[Id,Name]" \
  --output table

terraform -chdir=infra import \
  'module.cdn.aws_cloudfront_origin_access_control.frontend' <OAC_ID>
```

Para RDS:

```
terraform -chdir=infra import module.rds.aws_db_instance.this docuprocess-dev-mysql
```

Después de cada import, ejecuta `terraform -chdir=infra plan -refresh=true` y confirma que el plan solo crea lo que falta. Si el recurso no pertenece al proyecto, elimínalo con la consola o con AWS CLI y vuelve a aplicar.

7.4 Acciones denegadas por la SCP

Error	Fallback
`cloudtrail:CreateTrail`	`cloudtrail_mode = "event-history"` ; verificar con `verify-cloudtrail.sh`
`s3:GetBucketObjectLockConfiguration`	El módulo ya usa `awscc_s3_bucket` ; no cambies a `aws_s3_bucket`
`autoscaling:StartInstanceRefresh`	No usar `instance_refresh` ; reemplazar con ASG nuevo o terminación controlada
`dynamodb:TransactWriteItems` o acciones de ítem	`enable_dynamodb_transactions = false`
`dynamodb:UpdateContinuousBackups`	`enable_point_in_time_recovery = false`
`iam:CreateRole` / `iam:GetRole`	No se crean roles; se usa el instance profile preexistente

> `dynamodb:TransactWriteItems` no es una acción IAM válida por sí sola; la SCP deniega las acciones de escritura de ítems que usa la transacción. El fallback correcto es desactivar la transacción en la aplicación, no añadir esa cadena a una política.

7.5 Aplicación no responde o instancias unhealthy

Diagnóstico en orden:

```
# 1. ¿El ALB responde?
ALB_DNS="$(terraform -chdir=infra output -raw alb_dns_name)"
curl -v "http://$ALB_DNS/api/health"

# 2. ¿El target group ve las instancias?
TG_ARN="$(aws elbv2 describe-target-groups --names docuprocess-dev-app \
  --query 'TargetGroups[0].TargetGroupArn' --output text)"
aws elbv2 describe-target-health --target-group-arn "$TG_ARN" --output table

# 3. ¿Qué dice el userData?
# Usa los comandos de §5.6 y revisa /var/log/docuprocess-userdata.log.
```

Causas típicas:

- El artefacto no existe o el hash no coincide: vuelve a ejecutar `package-app.sh` y aplica.
- Node.js no se instaló: revisa el gateway endpoint de S3 y el log de `dnf`.
- Los parámetros SSM no existen o las instancias no tienen permiso para leerlos: revisa `aws ssm get-parameter --name /docuprocess-dev/db_password --with-decryption` desde una sesión autorizada.
- El security group de la app no permite el puerto 8000 desde el ALB: revisa `infra/modules/network/`.
- El servicio arrancó antes de que RDS estuviera disponible: el userData tiene un readiness temporal; si falló, reinicia `docuprocess-api` y revisa `journalctl`.

7.6 Documento atascado en `PROCESSING`

1. Verifica que el worker esté activo en **todas** las instancias (§5.6).
2. Revisa los logs:

```
sudo journalctl -u docuprocess-worker -n 200 --no-pager
```

1. Confirma que el objeto existe en S3. Usa el campo `key` que devolvió `POST /api/documents` al crear la subida:

```
DOCS_BUCKET="$(terraform -chdir=infra output -raw docs_bucket_name)"
KEY="<valor de key que devolvió POST /api/documents>"
aws s3api head-object --bucket "$DOCS_BUCKET" --key "$KEY"
```

1. Si el worker estaba caído y el documento quedó en `PROCESSING`, vuelve a ponerlo en `PENDING` para que el poll lo recoja:

```
aws dynamodb update-item \
  --table-name "$(terraform -chdir=infra output -raw dynamodb_table_name)" \
  --key "{\"documentId\":{\"S\":\"<DOCUMENT_ID>\"}}" \
  --update-expression 'SET #s = :p' \
  --expression-attribute-names '{"#s":"status"}' \
  --expression-attribute-values '{":p":{"S":"PENDING"}}'
```

1. Si vuelve a fallar, revisa el evento `FAILED` y el mensaje del worker. Un PDF corrupto debe terminar en `FAILED`, no quedarse en `PROCESSING`.

> El worker sondea cada 5 segundos y procesa lotes de 10. Un documento puede tardar unos segundos; si pasan más de 5 minutos, algo está mal.

7.7 Upload prefirmado con 403 o error de CORS

- La URL prefirmada expira a los **15 minutos**. Genera una nueva con `POST /api/documents`.
- El `Content-Type` del `PUT` debe ser exactamente el mismo que enviaste al crear el documento.
- El bucket de documentos debe tener CORS para el origen del frontend. Revisa:

```
aws s3api get-bucket-cors \
  --bucket "$(terraform -chdir=infra output -raw docs_bucket_name)"
```

- Si pruebas desde `localhost:5500`, el proyecto ya incluye ese origen. Si usas otro puerto, agrégalo en `cors_allowed_origins` y aplica.
- Un `403` con `SignatureDoesNotMatch` suele indicar que se modificó la URL o el header después de firmarla.

7.8 SNS no envía correos

1. Confirma que la suscripción no esté en `PendingConfirmation` (§4.7).
2. Revisa la política del topic: debe permitir `sns:Publish` al rol de la aplicación y a `cloudwatch.amazonaws.com` con `aws:SourceAccount`.

3. Verifica que el worker haya publicado el evento:

```
sudo journalctl -u docuprocess-worker -n 100 --no-pager | grep -i sns
```

1. Para alarmas, revisa que la acción `alarm` y `ok` apunte al topic correcto:

```
aws cloudwatch describe-alarms \
  --alarm-name-prefix docuprocess-dev- \
  --query 'MetricAlarms[].{Name:AlarmName,Actions:AlarmActions,OKActions:OKActions}' \
  --output json
```

1. Si el correo llegó a spam, marca “No es spam” antes de la demo.

7.9 Destrucción incompleta

Si `terraform destroy` deja recursos:

```
terraform -chdir=infra state list
```

- Buckets: vaciar con `./scripts/empty-bucket.sh`.
- RDS: verificar que no tenga deletion protection (el proyecto la desactiva) y que sea `available` antes de borrar.
- CloudFront: la distribución debe estar `Deployed`; el borrado puede tardar varios minutos.
- Parámetros SSM: se borran con el estado; si quedaron huérfanos, `aws ssm delete-parameter`.
- OAC: se borra después de la distribución; si queda, `aws cloudfront delete-origin-access-control`.

No borres el estado hasta que `terraform state list` esté vacío o solo contenga recursos que decidiste conservar.

8. Contraste con el enunciado y la rúbrica

8.1 Entregables del enunciado (§5)

Entregable	Dónde está	Estado
Repositorio Git con IaC modular, backend, worker, frontend y scripts userData	`infra/`, `app/`, `frontend/`, `scripts/`	Cumple
Documentación técnica: diagrama detallado	`README.md` (diagrama de componentes) y `docs/architecture.md` §1	Cumple
Justificación RDS vs. DynamoDB	`docs/rds-vs-dynamodb.md`	Cumple
Guía paso a paso de despliegue automatizado y solución de problemas en el Sandbox	Este documento (`.md` y `.pdf`) y la guía rápida `docs/guias/despliegue.md`	Cumple
Sustentación/demo: despliegue desde cero, flujo S3→worker→DynamoDB→SNS y carga del ASG	`docs/sustentacion.md`, `docs/informe-c4.md`, `docs/informe-a8.md`, `docs/informe-b10.md`	Cumple

8.2 Matriz requisito → implementación → evidencia

#	Requisito del enunciado	Implementación / evidencia	Estado
1	VPC con subredes públicas (ALB) y privadas (app + datos)	`infra/modules/network/`; VPC `10.0.0.0/16`, 2 públicas y 2 privadas; apply verificado	Verificado
2	ALB + ASG 2–4 con `t3.micro`	`infra/modules/compute/`; ASG 2/2 healthy; A8: 2→4 bajo carga	Verificado
3	API y worker en las mismas instancias	Unidades `docuprocess-api` y `docuprocess-worker` activas en ambas instancias	Verificado
4	SSM Session Manager, sin SSH ni bastión	Instance profile preexistente; sesiones y `send-command` probados	Verificado
5	RDS MySQL Single-AZ `db.t3.micro` para cuentas y autenticación	`infra/modules/rds/`; RDS `available`, 20 GB `gp2`	Verificado
6	S3 para archivos	Buckets docs/frontend/artifacts con PAB y SSE; CORS en docs; OAC en frontend; PUT prefirmado 200	Verificado
7	DynamoDB para metadatos, historial y resultados	Tablas `documents` y `document_events` `ACTIVE`; eventos `PENDING→PROCESSING→DONE`	Verificado
8	SNS para notificaciones	Topic `docuprocess-dev-alerts`; suscripción confirmada; `Delivered=3`, `Failed=0`	Verificado
9	CloudWatch: CPU de EC2 y métricas de RDS	4 alarmas con acciones `alarm` / `ok` hacia SNS; evidencia B10	Verificado
10	CloudTrail para auditoría	Modo `event-history` (90 días); `verify-cloudtrail.sh` OK	Verificado con salvedad: sin trail persistente por SCP
11	IaC modular (Terraform)	8 módulos; apply desde estado vacío; plan actual ~73 recursos	Verificado
12	Sin `CreateRole` / `GetRole`; solo roles preexistentes	`data.aws_iam_instance_profiles` + políticas basadas en recursos	Verificado
13	EC2 `t2/t3`, máximo 9 concurrentes, EBS `gp2` ≤ 35 GB	`t3.micro`, ASG 2–4, 20 GB `gp2`, validaciones	Verificado

#	Requisito del enunciado	Implementación / evidencia	Estado
14	RDS `gp2` ≤ 100 GB, Single-AZ	`db.t3.micro`, 20 GB, `MultiAZ=False`	Verificado
15	Comparativa RDS vs. DynamoDB	`docs/rds-vs-dynamodb.md`	Verificado
16	Diagrama detallado y flujo	`README.md` y `docs/architecture.md` §1	Verificado
17	Guía de despliegue y troubleshooting	Este documento (`.md` / `.pdf`) + `docs/guias/despliegue.md`	Verificado
18	Demo: S3 → worker → DynamoDB → SNS	E2E 13-nov-2026: PDF `DONE`, PNG `DONE`, corrupto `FAILED`, documento ajeno 404, correo SNS	Verificado
19	Prueba de carga para el ASG	A8: 600 s, 50 concurrentes, CPU 98–99 %, ASG 2→4, 5.848 requests	Verificado

> La fila 11 conserva el "68" histórico en `docs/trazabilidad.md` porque corresponde al apply de la integración anterior. El plan actual, con el módulo `monitoring`, es del orden de 73 recursos; la diferencia no es un fallo, es la evolución del proyecto.

8.3 Mapeo a la rúbrica

Criterio	Peso	Cómo se cubre	Evidencia
IaC y automatización	30 %	Terraform modular, variables validadas, outputs, userData, scripts, CI de validación	`infra/`, `scripts/`, `.github/workflows/ci.yml`
Arquitectura e integración	25 %	VPC segmentada, ALB/ASG, CloudFront+OAC, endpoints, SSM, S3/DynamoDB/RDS/SNS	`docs/architecture.md`, este documento §5
Gestión de datos y persistencia	20 %	RDS para usuarios, DynamoDB para metadatos/eventos, S3 para archivos, prefirmadas	`docs/rds-vs-dynamodb.md`, §5.5
Monitoreo, alertas y resiliencia	15 %	4 alarmas, SNS, CloudTrail, health/ready, ASG, reemplazo controlado	`docs/informe-b10.md`, §5.7, §5.8
Documentación y sustentación	10 %	README, arquitectura, trazabilidad, esta guía y guion de demo	`docs/sustentacion.md`, este documento

8.4 Restricciones: cumplimiento y salvedades

Restricción	Cumplimiento	Salvedad honesta
IAM solo lectura; sin `CreateRole` / `GetRole`	Cumple	Se depende de un rol preexistente del laboratorio
Lambda no disponible	Cumple	Worker en EC2, no serverless
EC2 `t2/t3` y máximo 9 concurrentes	Cumple	Durante un reemplazo puede haber instancias temporales; no se supera el límite con 2–4 deseadas
EBS `gp2` ≤ 35 GB	Cumple	20 GB por instancia
RDS `db.t2/t3`, Single-AZ, `gp2` ≤ 100 GB	Cumple	Sin alta disponibilidad real; es una limitación del Sandbox
Sin Route 53 ni dominio propio	Cumple	Se usa el dominio por defecto de CloudFront
Terraform o CloudFormation	Cumple	Terraform
Sin SSH ni bastión	Cumple	Solo SSM Session Manager

Restricción	Cumplimiento	Salvedad honesta
CloudTrail	Cumple con salvedad	`event-history`; el Sandbox deniega `CreateTrail`
SNS por correo	Cumple con salvedad	La confirmación del buzón es manual y no la puede hacer Terraform

8.5 Brechas y riesgos aceptados

Brecha / riesgo	Impacto	Mitigación	Estado
No hay CD a AWS	El `apply` es manual	CI valida; el despliegue se ejecuta con credenciales del Lab	Aceptado por restricción
CloudTrail sin trail persistente	Auditoría limitada a 90 días	`event-history` + `verify-cloudtrail.sh`	Aceptado por SCP
SNS requiere confirmación manual	Sin correo no hay alertas	Confirmar antes de la demo; checklist §5.1	Aceptado
ASG sin `instance_refresh`	Reemplazo menos elegante	ASG nuevo o terminación controlada	Aceptado por SCP
Scale-in lento (~15 min)	La demo puede terminar con 4 instancias	Esperar o explicar la alarma de baja	Documentado
Estado de Terraform local	No se comparte entre integrantes	Un responsable del estado; no commitear `tfstate`	Aceptado
Secretos en el estado local	Riesgo si se comparte	Borrar estado tras destroy; no subirlo	Aceptado
RDS Single-AZ	Sin failover automático	Backups y reconstrucción; es requisito del Sandbox	Aceptado
504 transitorio en CloudFront	Falla durante reemplazo/ draining	Reintentar; no probar durante reemplazos	Documentado
4 respuestas no-2xx en A8	Ruido durante el escalado	Se reporta en la evidencia; no se oculta	Documentado

8.6 Checklist de aceptación

- ☐ El repositorio clona y `terraform validate` pasa.
- ☐ `terraform apply` termina desde un estado vacío (o desde huérfanos importados).
- ☐ El plan actual ronda los 73 recursos y no destruye recursos ajenos.
- ☐ `app_url` sirve el frontend y `/api/health` / `/api/ready` responden 200.
- ☐ El target group está 2/2 healthy y el ASG 2/2 InService.
- ☐ Registro, login, creación de documento y upload prefirmado funcionan.
- ☐ Un PDF termina `DONE`; un corrupto termina `FAILED`.
- ☐ DynamoDB contiene el documento y sus eventos.
- ☐ SNS está confirmado y llega el correo del documento.
- ☐ Las 4 alarmas están en `OK` y tienen acciones hacia SNS.
- ☐ CloudTrail muestra eventos recientes.
- ☐ La prueba de carga lleva el ASG de 2 a 4.
- ☐ `terraform destroy` deja el laboratorio sin recursos del proyecto.
- ☐ Esta guía y su PDF están enlazados desde README y trazabilidad.

9. Anexos

A. Outputs de Terraform

Output	Ejemplo de uso
`account_id`	Confirmar la cuenta antes de aplicar
`app_url`	Abrir la demo
`alb_dns_name`	Diagnóstico directo contra el ALB
`asg_name`	Capacidad y reemplazo de instancias
`sns_topic_arn`	Suscripción y alarmas
`rds_endpoint`	Verificar RDS
`docs_bucket_name`	Revisar objetos `uploads/`
`frontend_bucket_name`	Verificar el build publicado
`artifacts_bucket_name`	Verificar `releases/app-<hash>.zip`
`cloudfront_distribution_id`	Invalidaciones manuales
`cloudfront_domain_name`	Dominio por defecto
`cloudtrail_audit_mode`	Confirmar `event-history`
`alarm_names`	Lista de las 4 alarmas
`instance_profile_name`	Confirmar el rol SSM

B. Variables de Terraform

Variable	Tipo	Default	Nota
`aws_region`	string	`us-east-1`	No cambiar en el Sandbox
`aws_profile`	string	`null`	Opcional; si se usa, `academy`
`project_name`	string	`docuprocess`	Prefijo de nombres
`environment`	string	`dev`	Prefijo de nombres
`azs`	list(string)	`us-east-1a` , `us-east-1b`	Dos AZ
`vpc_cidr`	string	`10.0.0.0/16`	Red del proyecto
`public_subnet_cidrs`	list(string)	`10.0.1.0/24` , `10.0.2.0/24`	ALB
`private_subnet_cidrs`	list(string)	`10.0.11.0/24` , `10.0.12.0/24`	App y datos
`enable_nat_gateway`	bool	`false`	Se usan endpoints
`enable_interface_endpoints`	bool	`true`	SSM, SNS y otros
`instance_type`	string	`t3.micro`	Límite del Sandbox
`instance_role_name`	string	`AmazonSSMRoleForInstancesQuickSetup`	Rol preexistente
`asg_min_size`	number	`2`	Mínimo de la demo
`asg_max_size`	number	`4`	Máximo de la demo

Variable	Tipo	Default	Nota
`asg_desired_capacity`	number	`2`	Estado inicial
`cpu_target_value`	number	`60`	Target tracking
`root_volume_size`	number	`20`	`gp2`, ≤ 35 GB
`db_name`	string	`docuprocess`	Base MySQL
`db_username`	string	`docuadmin`	Usuario MySQL
`db_password`	string	—	Obligatoria; `openssl rand -hex 16`
`jwt_secret`	string	—	Obligatoria; `openssl rand -hex 32`
`cors_allowed_origins`	list(string)	localhost:5500 y 127.0.0.1:5500	Agregar el origen real si cambia
`dynamodb_table_name`	string	`documents`	Metadatos
`document_events_table_name`	string	`document_events`	Historial
`enable_dynamodb_transactions`	bool	`true`	Fallback `false`
`worker_poll_seconds`	number	`5`	Sondeo del worker
`worker_batch_size`	number	`10`	Lote
`upload_timeout_minutes`	number	`15`	Expiración de la URL
`max_file_mb`	number	`10`	Tamaño máximo
`enable_point_in_time_recovery`	bool	`true`	Fallback `false`
`alert_email`	string	—	Obligatoria
`force_destroy_buckets`	bool	`true`	Limpieza del laboratorio
`cloudtrail_mode`	string	`event-history`	`trail` no disponible
`docs_lifecycle_expiration_days`	number	`0`	`0` desactiva

C. Plantilla de evidencia para la entrega

```
Fecha (UTC):
Cuenta / región:
Rama y commit:
Resultado de terraform apply (recursos):
Estado del ASG (desired / in-service / healthy):
Estado del target group:
Resultado de /api/health y /api/ready:
Documento PDF: documentId / estado / páginas / SHA-256 / bytes
Documento corrupto: estado / mensaje
Correo SNS: recibido (sí/no) / hora
Alarmas: nombres / estado / acciones
CloudTrail: eventos consultados / modo
Prueba de carga: duración / concurrencia / requests / errores / CPU / ASG
Incidencias y solución:
```

D. Enlaces útiles

- Guía rápida: `despliegue.md`

- Credenciales de AWS Academy: `aws-academy-credenciales.md`
- Pruebas de storage: `pruebas-storage.md`
- Arquitectura e integración: `../architecture.md`
- RDS vs. DynamoDB: `../rds-vs-dynamodb.md`
- Trazabilidad: `../trazabilidad.md`
- Sustentación: `../sustentacion.md`
- Informe de monitoreo: `../informe-b10.md`
- Informe de carga: `../informe-a8.md`
- Informe de despliegue desde cero: `../informe-c4.md`

E. Glosario

Término	Definición
ALB	Application Load Balancer; recibe el tráfico HTTP de la API
ASG	Auto Scaling Group; mantiene 2–4 instancias y escala con CPU
OAC	Origin Access Control; permite que CloudFront lea el bucket privado
Preflight	Verificación previa de credenciales, cuenta, región, estado y huérfanos
SCP	Service Control Policy; límites de permisos del Sandbox
SSM	AWS Systems Manager; se usa para sesiones y comandos sin SSH
Target group	Grupo de destinos del ALB; aquí, las instancias en el puerto 8000

Colofón. Documento generado a partir del repositorio `docuprocess-cloud` en la rama de la guía, con evidencia viva del 13 de noviembre de 2026. El PDF se produjo con WeasyPrint 70.0 y una hoja de estilos de impresión A4; no se versiona el generador para mantener el repositorio liviano. Si se actualiza la guía, vuelve a generar el PDF y revisa los enlaces y las cifras de recursos.